Jordan Forbes
ECE-GY 9953 Gen AI Based Chip Design
9/22/26

1. Binary_to_bcd
    a. This example converts a 5-bit binary input to 8-bit BCD. This is the only example where all the test cases passed on the first run.
2. Sequence Detector
    a. This example detects an 8-step sequence of 3 bit values. There the issues started. The LLM kept failing at cycle 8 and it was blaming the testbench and timing. This is the only example that was left unresolved
3. Shift Register
    a. This was a 8-bit serial in, parallel out shift register. The first run gave the output “xxxxxxxx”. The main issue was the test bench. The testbench checked the output before it could ever be reset so it couldn't hold the correct yet. Then the issue was a naming mismatch. The LLM suggested reset and initialization fixes. After fixing the testbench all of the tests passed
4. Abro State Machine
    a. For this example there was a naming mismatch leading to the testbench not running. A wrapper was added at the end of the design file allowing the testbench to run. Then the test cases were failing. The LLM gave suggestions on editing the testbench to get the desired output.
5. Dice Roller
    a. This example used an 8-bit shift register, and the values rolled were based on a 2-bit select. The main issue with this one was a port mismatch with “rst_n”. After failing the test cases initially I gave the problem to the LLM and it renamed the port. After fixing the port issue in the design code all the test cases passed
6. LFSR
    a. This example is an 8-bit Linear Feedback Shift Register with an initial state of 10001010 and taps at 1,4,6, and 7. The issue with this one was mainly the designs the LLM came up with. It kept declaring a feedback variable inside of an always block which isn't allowed. The testbench then reported a mismatch that the LLM had a hard time tracing and finding the issue. A wrapper had to be used in this one as well computing the feedback inline treating the taps as indexed allowing all the tests to pass
7. Sequence Generator
    a. Outputs the 8-bit sequence AF, BC, E2, 78, FF, E2, 0B, 8D and repeats it while enabled. The issue with this one was that the sequence generator

would not disable causing the last test case to fail. The cause turned out to be the else statement in the design code. when enable goes low, the else branch forces data to 0. The testbench expected the output to hold the value but it wasn't. After telling this to the LLMit made the change and the sequence disabled and all the tests passed

8. Traffic Light
    a. This example was a traffic light Finite State Machine. It resets to red, turns green after 32 clock cycles, yellow after 20 and back to red after 7. The issue with this one is the LLM used a typdef enum which gave a syntax error on line 12 of the code. After bringing it to the LLMs attention it switched to localparam which went straight to the cause. This allowed all 4 tests to pass
9. CPU/ 8-bit Processor
    a. This was an accumulator cpu that must fit within 100 standard cells with single byte instructions. After entering the first prompt the LLM laid out the whole plan to create the CPU including the Architecture overview, Basic components, Design Phases, and finally Prototyping and Testing. I then told it to create the design based on the plan it laid out. It then created the detailed plan that included things like what registers that are going to be used, the ISA, and the functional blocks etc. This also had its own issues. After running the code on the testbench I created for it, the last two test cases failed. After telling the LLM the issues it debugged its own code and then all the test cases passed.

Many of the failures came from mismatches between the generated design and the testbench, such as differing port or module names, or syntax that iverilog didn't support without SystemVerilog enabled. Since the LLM couldn't see the testbenches, it had to guess at the expected interface and behavior. Follow-up prompts sometimes fixed an issue quickly, but other times it took several rounds. The LLM often gave generic debugging advice or revised its own earlier code instead of the code I was actually running. Being as specific as possible helped a lot: including the exact error message, the relevant lines of code, and the testbench's expected port names. Without that context, the LLM often couldn't pinpoint the main issue on its own.